Ccsds Parameter Table Router

Component Design Document

1 Description

This component receives segmented CCSDS packets containing parameter tables, reassembles them in a staging buffer, and routes completed tables to downstream components that accept a Parameters_Memory_Region type. It supports loading parameter tables from a designated component on command, which will often be a Parameter_Store sitting in front of persistent storage.
A generator exists to produce the routing table that maps parameter table IDs to output connector indexes. See the gen/ subdirectory for documentation.

2 Requirements

The requirements for the CCSDS Parameter Table Router component are specified below.

1. The component shall receive segmented CCSDS packets, reassemble them in an internal staging buffer, and extract the parameter table ID from the first two bytes of the first segment.
2. The component shall route completed parameter tables to one or more downstream destinations based on a routing table provided during initialization.
3. During table distribution, the component shall send to non-load_from destinations first, followed by the load_from destination last, to avoid persisting an invalid table.
4. If any downstream destination fails or times out during table distribution, the component shall abandon remaining destinations and report the failure via event.
5. The component shall include a command to load a single parameter table from its load_from source and distribute it to all other destinations.
6. The component shall include a command to load all parameter tables that have a load_from source configured.
7. The component shall be configurable to automatically load all parameter tables from persistent storage during Set_Up.
8. The component shall use a configurable tick-based timeout when waiting for responses from downstream components.
9. The component shall report packet reception counts, rejection counts, table update counts, and last table received information as data products.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *active*
- **Number of Connectors** - 9
- **Number of Invokee Connectors** - 4
- **Number of Invoker Connectors** - 5
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - 1
- **Number of Commands** - 2
- **Number of Parameters** - *None*
- **Number of Events** - 18
- **Number of Faults** - *None*
- **Number of Data Products** - 5
- **Number of Data Dependencies** - *None*
- **Number of Packets** - *None*

3.2 Diagram

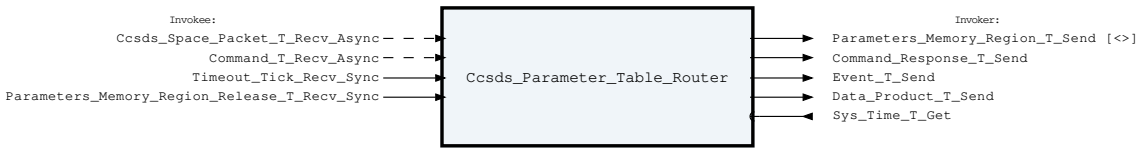

Figure 1: Ccsds Parameter Table Router component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Ccsds Parameter Table Router Invokee Connectors

Name	Kind	Type	Return_Type	Count
Ccsds_Space_Packet_T_Recv_Async	recv_async	Ccsds_Space_Packet.T	-	1
Command_T_Recv_Async	recv_async	Command.T	-	1
Timeout_Tick_Recv_Sync	recv_sync	Tick.T	-	1
Parameters_Memory_Region_Release_T_Recv_Sync	recv_sync	Parameters_Memory_Region_Release.T	-	1

Connector Descriptions:

- **Ccsds_Space_Packet_T_Recv_Async** - Receives segmented CCSDS packets containing parameter table data.
- **Command_T_Recv_Async** - The command receive connector.
- **Timeout_Tick_Recv_Sync** - Periodic tick used for timeout counting when waiting for downstream responses.
- **Parameters_Memory_Region_Release_T_Recv_Sync** - Synchronous response from downstream components after a Set or Get operation.

3.3.2 Internal Queue

This component contains an internal first-in-first-out (FIFO) queue to handle asynchronous messages. This queue is sized at initialization as a configurable number of bytes. Determining the size of the component queue can be difficult. The following table lists the connectors that will put asynchronous messages onto the queue, and the maximum sizes of each of those messages on the queue. Note that each message put onto the queue also incurs an overhead on the queue of 5 additional bytes, which is included in the max message size below:

Table 2: Ccsds Parameter Table Router Asynchronous Connectors

Name	Type	Max Size (bytes)
Ccsds_Space_Packet_T_Recv_Async	Ccsds_Space_Packet.T	1285
Command_T_Recv_Async	Command.T	265

If you are unsure how to size the queue of this component, it is recommended that you make the queue size a multiple of the largest size found above.

3.3.3 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 3: Ccsds Parameter Table Router Invoker Connectors

Name	Kind	Type	Return_Type	Count
Parameters_Memory_Region_T_Send	send	Parameters_Memory_Region.T	-	<>
Command_Response_T_Send	send	Command_Response.T	-	1
Event_T_Send	send	Event.T	-	1
Data_Product_T_Send	send	Data_Product.T	-	1
Sys_Time_T_Get	get	-	Sys_Time.T	1

Connector Descriptions:

- **Parameters_Memory_Region_T_Send** - Arrayed output connector for sending parameter table memory regions to downstream components.
- **Command_Response_T_Send** - Send command responses.
- **Event_T_Send** - Events are sent out of this connector.
- **Data_Product_T_Send** - Data products are sent out of this connector.
- **Sys_Time_T_Get** - The system time is retrieved via this connector.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component achieves base class initialization using the `init_Base` subprogram. This subprogram requires the following parameters:

Table 4: Ccsds Parameter Table Router Base Initialization Parameters

Name	Type
`Parameters_Memory_Region_T_Send_Count`	`Connector_Count_Type`
`Queue_Size`	`Natural`

Parameter Descriptions:

- **`Parameters_Memory_Region_T_Send_Count`** - The size of the Parameters_Memory_Region_T_Send invoker connector array.
- **`Queue_Size`** - The number of bytes that can be stored in the component's internal queue.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 5: Ccsds Parameter Table Router Set Id Bases Parameters

Name	Type
`Command_Id_Base`	`Command_Types.Command_Id_Base`
`Data_Product_Id_Base`	`Data_Product_Types.Data_Product_Id_Base`
`Event_Id_Base`	`Event_Types.Event_Id_Base`

Parameter Descriptions:

- **`Command_Id_Base`** - The value at which the component's command identifiers begin.
- **`Data_Product_Id_Base`** - The value at which the component's data product identifiers begin.
- **`Event_Id_Base`** - The value at which the component's event identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initialization parameters for the Parameter Table Router. The `init` subprogram requires the following parameters:

Table 6: Ccsds Parameter Table Router Implementation Initialization Parameters

Name	Type	Default Value
`Table`	`Ccsds_Parameter_Table_Router_Types.Router_Table`	*None provided*
`Buffer_Size`	`Positive`	*None provided*
`Ticks_Until_Timeout`	`Natural`	*None provided*
`Load_All_Parameter_Tables_On_Set_Up`	`Boolean`	`False`

Parameter Descriptions:

- **`Table`** - The routing table mapping parameter table IDs to destination connector indexes. Typically produced by the generator.
- **`Buffer_Size`** - The size in bytes of the internal staging buffer for reassembling segmented CCSDS packets.
- **`Ticks_Until_Timeout`** - The number of timeout ticks to wait for a response from a downstream component before declaring a timeout.
- **`Load_All_Parameter_Tables_On_Set_Up`** - If True, all parameter tables that have a load_from source will be loaded from persistent storage during Set_Up.

3.6 Commands

Commands for the Parameter Table Router component.

Table 7: Ccsds Parameter Table Router Commands

Local ID	Command Name	Argument Type
0	`Load_Parameter_Table`	`Parameter_Table_Id.T`
1	`Load_All_Parameter_Tables`	-

Command Descriptions:

- **`Load_Parameter_Table`** - Load a single parameter table from its load_from source and distribute to other destinations.
- **`Load_All_Parameter_Tables`** - Load all parameter tables that have a load_from source configured and distribute to their destinations.

3.7 Parameters

The Ccsds Parameter Table Router component has no parameters.

3.8 Events

Events for the Parameter Table Router component.

Table 8: Ccsds Parameter Table Router Events

Local ID	Event Name	Parameter Type
0	Receiving_New_Table	Parameter_Table_Id.T
1	Table_Received	Parameter_Table_Id.T
2	Table_Updated	Parameter_Table_Id.T
3	Loading_Table	Parameter_Table_Id.T
4	Table_Loaded	Parameter_Table_Id.T
5	Table_Update_Failure	Parameter_Table_Operation_ Failure_Info.T
6	Table_Update_Timeout	Parameter_Table_Timeout_Info.T
7	Table_Load_Failure	Parameter_Table_Operation_ Failure_Info.T
8	No_Load_Source	Parameter_Table_Id.T
9	Unrecognized_Table_Id	Parameter_Table_Id.T
10	Packet_Ignored	Ccsds_Primary_Header.T
11	Too_Small_Table	Ccsds_Primary_Header.T
12	Staging_Buffer_Overflow	Ccsds_Primary_Header.T
13	Invalid_Command_Received	Invalid_Command_Info.T
14	Command_Dropped	Command_Header.T
15	Packet_Dropped	Ccsds_Primary_Header.T
16	Loading_All_Parameter_Tables	-
17	All_Parameter_Tables_Loaded	-

Event Descriptions:

- **Receiving_New_Table** - A new parameter table FirstSegment has been received and buffering has started.
- **Table_Received** - A complete parameter table has been reassembled from CCSDS segments.
- **Table_Updated** - A parameter table has been successfully sent to all downstream destinations.
- **Loading_Table** - Starting to load a parameter table from its load_from source.
- **Table_Loaded** - A parameter table has been successfully loaded from persistent storage and sent to all destinations.
- **Table_Update_Failure** - A downstream component rejected a parameter table update. Includes the table ID, connector index, and release status.
- **Table_Update_Timeout** - Timed out waiting for a response from a downstream component during table update.
- **Table_Load_Failure** - Failed to load a parameter table from persistent storage. Includes the table ID, connector index, and release status.
- **No_Load_Source** - Load_Parameter_Table command received for a table ID that has no load_from destination configured.
- **Unrecognized_Table_Id** - Received a parameter table with a Table ID not found in the routing table.
- **Packet_Ignored** - A CCSDS packet was ignored (continuation/last segment without prior first segment, or unsegmented).
- **Too_Small_Table** - A FirstSegment packet was too small to contain a Table ID (less than 2 bytes of data).
- **Staging_Buffer_Overflow** - The staging buffer is full. Dropping packets until a new FirstSegment resets the buffer.
- **Invalid_Command_Received** - A command was received with invalid parameters.
- **Command_Dropped** - A command was dropped due to a full queue.

- **Packet_Dropped** - A CCSDS packet was dropped due to a full queue.
- **Loading_All_Parameter_Tables** - Starting to load all parameter tables that have a load_-from source.
- **All_Parameter_Tables_Loaded** - Finished loading all parameter tables.

3.9 Data Products

Data products for the Parameter Table Router component.

Table 9: Ccsds Parameter Table Router Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Num_Packets_Received	Packed_U32.T
0x0001 (1)	Num_Packets_Rejected	Packed_U32.T
0x0002 (2)	Num_Tables_Updated	Packed_U32.T
0x0003 (3)	Num_Tables_Invalid	Packed_U32.T
0x0004 (4)	Last_Table_Received	Parameter_Table_Received_Info.T

Data Product Descriptions:

- **Num_Packets_Received** - Total number of CCSDS packets received on the async connector.
- **Num_Packets_Rejected** - Number of packets rejected (ignored, too small, or buffer overflow).
- **Num_Tables_Updated** - Number of parameter tables successfully distributed to all destinations.
- **Num_Tables_Invalid** - Number of parameter tables that failed distribution, had unrecognized IDs, or timed out.
- **Last_Table_Received** - Information about the last complete parameter table received.

3.10 Data Dependencies

The Ccsds Parameter Table Router component has no data dependencies.

3.11 Packets

The Ccsds Parameter Table Router component has no packets.

3.12 Faults

The Ccsds Parameter Table Router component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Ccsds_Parameter_Table_Router_Tests* Test Suite

Unit tests for the Parameter Table Router component.

Test Descriptions:

- **`Test_Init`** - Verify Init populates binary tree, creates staging buffer, and validates routing table.
- **`Test_Set_Up`** - Verify Set_Up publishes initial data product values.
- **`Test_Set_Up_Load_All`** - Init with Load_All_Parameter_Tables_On_Set_Up. Verify tables are loaded during Set_Up.
- **`Test_Nominal_Segmented_Upload`** - Send First + Continuation + Last for a known table ID. Verify events and data products.
- **`Test_Unsegmented_Upload`** - Send a single Unsegmented packet for a known table ID. Verify events and data products.
- **`Test_Multi_Destination_Order`** - Verify non-Load_From destinations sent first in YAML order, Load_From last.
- **`Test_Packet_Ignored`** - Continuation and Last without prior First produce Packet_Ignored events.
- **`Test_Too_Small_Table`** - FirstSegment with less than 2 bytes produces Too_Small_Table event.
- **`Test_Buffer_Overflow`** - Data exceeding buffer capacity produces Staging_Buffer_Overflow event.
- **`Test_Unrecognized_Table_Id`** - Complete table with unknown ID produces Unrecognized_Table_Id event and increments invalid counter.
- **`Test_Destination_Failure`** - Downstream returns failure status. Verify Table_Update_-Failure event with correct info.
- **`Test_Destination_Timeout`** - Downstream does not respond. Verify Table_Update_-Timeout event.
- **`Test_Partial_Failure_Stops`** - Multi-destination table where first destination fails. Remaining destinations not sent to.
- **`Test_Load_Table_Nominal`** - Load a table from its load_from source. Verify Loading_-Table and Table_Loaded events.
- **`Test_Load_Table_No_Load_Source`** - Load command for table with no load_from. Verify No_Load_Source event and command failure.
- **`Test_Load_Table_Unrecognized`** - Load command for unknown table ID. Verify event and command failure.
- **`Test_Load_Table_Failure`** - Load fails during Get or Set. Verify failure events and command failure.
- **`Test_Load_All_Nominal`** - Load all tables. Verify start/end events and per-table events.
- **`Test_Load_All_Partial_Failure`** - One table fails during load all. Verify command failure and other tables still attempted.
- **`Test_Packet_Dropped`** - Overflow CCSDS packet queue. Verify Packet_Dropped event.
- **`Test_Command_Dropped`** - Overflow command queue. Verify Command_Dropped event.
- **`Test_Invalid_Command`** - Send command with corrupted arguments. Verify Invalid_Command_Received event.
- **`Test_Load_From_Destination_Failure`** - Upload table where non-Load_From succeeds but Load_From (sent last) fails.
- **`Test_Load_Command_Set_Failure`** - Load command where Get succeeds but subsequent Set to non-Load_From fails.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Ccsds_Primary_Header.T:

Record for the CCSDS Packet Primary Header *Preamble (inline Ada definitions):*

```
subtype Three_Bit_Version_Type is Interfaces.Unsigned_8 range 0 .. 7;
type Ccsds_Apid_Type is mod 2**11;
type Ccsds_Sequence_Count_Type is mod 2**14;
```

Table 10: Ccsds_Primary_Header Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Version	Three_Bit_Version_Type	0 to 7	3	0	2
Packet_Type	Ccsds_Enums.Ccsds_Packet_Type.E	0 => Telemetry 1 => Telecommand	1	3	3
Secondary_Header	Ccsds_Enums.Ccsds_Secondary_Header_Indicator.E	0 => Secondary_Header_Not_Present 1 => Secondary_Header_Present	1	4	4
Apid	Ccsds_Apid_Type	0 to 2047	11	5	15
Sequence_Flag	Ccsds_Enums.Ccsds_Sequence_Flag.E	0 => Continuationsegment 1 => Firstsegment 2 => Lastsegment 3 => Unsegmented	2	16	17
Sequence_Count	Ccsds_Sequence_Count_Type	0 to 16383	14	18	31

Packet_ Length	Interfaces. Unsigned_ 16	0 to 65535	16	32	47

Field Descriptions:

- **Version** - Packet Version Number
- **Packet_Type** - Packet Type
- **Secondary_Header** - Does packet have CCSDS secondary header
- **Apid** - Application process identifier
- **Sequence_Flag** - Sequence Flag
- **Sequence_Count** - Packet Sequence Count
- **Packet_Length** - This is the packet data length. One added to this number corresponds to the number of bytes included in the data section of the CCSDS Space Packet.

Ccsds_Space_Packet.T:

Record for the CCSDS Space Packet *Preamble (inline Ada definitions):*

```
use Basic_Types;
subtype Ccsds_Data_Type is Byte_Array (0 ..
↪  Configuration.Ccsds_Packet_Buffer_Size - 1);
```

Table 11: Ccsds_Space_Packet Packed Record : 10240 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Ccsds_ Primary_ Header.T	-	48	0	47	–
Data	Ccsds_Data_ Type	-	10192	48	10239	Header. Packet_Length

Field Descriptions:

- **Header** - The CCSDS Primary Header
- **Data** - User Data Field

Command.T:

Generic command packet for holding arbitrary commands

Table 12: Command Packed Record : 2080 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Command_ Header.T	-	40	0	39	–

Arg_Buffer	Command_ Types. Command_Arg_ Buffer_Type	-	2040	40	2079	Header.Arg_ Buffer_Length

Field Descriptions:

- **Header** - The command header
- **Arg_Buffer** - A buffer that contains the command arguments

Command_Header.T:

Generic command header for holding arbitrary commands

Table 13: Command_Header Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types. Command_Source_Id	0 to 65535	16	0	15
Id	Command_Types. Command_Id	0 to 65535	16	16	31
Arg_Buffer_Length	Command_Types. Command_Arg_Buffer_ Length_Type	0 to 255	8	32	39

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Id** - The command identifier
- **Arg_Buffer_Length** - The number of bytes used in the command argument buffer

Command_Response.T:

Record for holding command response data.

Table 14: Command_Response Packed Record : 56 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_ Types.Command_ Source_Id	0 to 65535	16	0	15
Registration_ Id	Command_ Types.Command_ Registration_ Id	0 to 65535	16	16	31
Command_Id	Command_Types. Command_Id	0 to 65535	16	32	47

Status	Command_Enums. Command_ Response_ Status.E	0 => Success 1 => Failure 2 => Id_Error 3 => Validation_Error 4 => Length_Error 5 => Dropped 6 => Register 7 => Register_Source	8	48	55

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Registration_Id** - The registration ID. An ID assigned to each registered component at initialization.
- **Command_Id** - The command ID for the command response.
- **Status** - The command execution status.

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 15: Data_Product Packed Record : 344 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_ Header.T	-	88	0	87	–
Buffer	Data_Product_ Types.Data_ Product_ Buffer_Type	-	256	88	343	Header.Buffer_ Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 16: Data_Product_Header Packed Record : 88 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	64	79
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 32	8	80	87

Field Descriptions:

- **Time** - The timestamp for the data product item.

- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Event.T:

Generic event packet for holding arbitrary events

Table 17: Event Packed Record : 344 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Event_Header.T	-	88	0	87	–
Param_Buffer	Event_Types. Parameter_ Buffer_Type	-	256	88	343	Header.Param_ Buffer_Length

Field Descriptions:

- **Header** - The event header
- **Param_Buffer** - A buffer that contains the event parameters

Event_Header.T:

Generic event packet for holding arbitrary events

Table 18: Event_Header Packed Record : 88 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Event_Types.Event_ Id	0 to 65535	16	64	79
Param_Buffer_Length	Event_Types. Parameter_Buffer_ Length_Type	0 to 32	8	80	87

Field Descriptions:

- **Time** - The timestamp for the event.
- **Id** - The event identifier
- **Param_Buffer_Length** - The number of bytes used in the param buffer

Invalid_Command_Info.T:

Record for holding information about an invalid command

Table 19: Invalid_Command_Info Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Command_Types. Command_Id	0 to 65535	16	0	15
Errant_Field_ Number	Interfaces. Unsigned_32	0 to 4294967295	32	16	47
Errant_Field	Basic_Types.Poly_ Type	-	64	48	111

Field Descriptions:

- **Id** - The command Id received.
- **Errant_Field_Number** - The field that was invalid. 1 is the first field, 0 means unknown field, 2**32 means that the length field of the command was invalid.
- **Errant_Field** - A polymorphic type containing the bad field data, or length when Errant_-Field_Number is 2**32.

Memory_Region.T:

A memory region described by a system address and length (in bytes).

Table 20: Memory_Region Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Address	System.Address	-	64	0	63
Length	Natural	0 to 2147483647	32	64	95

Field Descriptions:

- **Address** - The starting address of the memory region.
- **Length** - The number of bytes at the given address to associate with this memory region.

Packed_U32.T:

Single component record for holding packed unsigned 32-bit value.

Table 21: Packed_U32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces. Unsigned_32	0 to 4294967295	32	0	31

Field Descriptions:

- **Value** - The 32-bit unsigned integer.

Parameter_Table_Id.T:

A packed record wrapping a parameter table ID.

Table 22: Parameter_Table_Id Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Table_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The parameter table ID.

Parameter_Table_Operation_Failure_Info.T:

Information about a failed parameter table operation (update or load).

Table 23: Parameter_Table_Operation_Failure_Info Packed Record : 136 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Table_Id	Parameter_Types. Parameter_Table_Id	0 to 65535	16	0	15
Connector_Index	Connector_Types. Connector_Index_ Type	1 to 65535	16	16	31
Release	Parameters_Memory_ Region_Release.T	-	104	32	135

Field Descriptions:

- **Table_Id** - The parameter table ID that the operation was attempted for.
- **Connector_Index** - The connector index of the destination that failed.
- **Release** - The release response from the downstream component containing the memory region and status.

Parameter_Table_Received_Info.T:

Information about the current or most recent parameter table operation.

Table 24: Parameter_Table_Received_Info Packed Record : 152 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Table_Id	Parameter_ Types. Parameter_ Table_Id	0 to 65535	16	0	15
Status	Ccsds_ Parameter_ Table_ Router_ Enums.Table_ Status.E	0 => Receiving_Table 1 => Table_Update_Success 2 => Table_Update_Failed 3 => Unrecognized_Id 4 => Buffer_Overflow 5 => Too_Small 6 => Packet_Ignored	8	16	23
Bytes_ Received	Natural	0 to 2147483647	32	24	55
Packets_ Received	Natural	0 to 2147483647	32	56	87
Timestamp	Sys_Time.T	-	64	88	151

Field Descriptions:

- **Table_Id** - The parameter table ID (0 if no table has been received).
- **Status** - The current status of the table operation.
- **Bytes_Received** - Number of table data bytes currently in the staging buffer.
- **Packets_Received** - Number of CCSDS packets received for the current table.
- **Timestamp** - The time of the most recent status update.

Parameter_Table_Timeout_Info.T:

Information about a parameter table operation that timed out waiting for a downstream response.

Table 25: Parameter_Table_Timeout_Info Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Table_Id	Parameter_Types. Parameter_Table_Id	0 to 65535	16	0	15
Connector_Index	Connector_Types. Connector_Index_ Type	1 to 65535	16	16	31

Field Descriptions:

- **Table_Id** - The parameter table ID that the operation was attempted for.
- **Connector_Index** - The connector index of the destination that timed out.

Parameters_Memory_Region.T:

A packed record which holds the parameter memory region to operate on as well as an enumeration specifying the operation to perform.

Table 26: Parameters_Memory_Region Packed Record : 104 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Region	Memory_Region.T	-	96	0	95
Operation	Parameter_Enums. Parameter_Table_ Operation_Type.E	0 => Get 1 => Set 2 => Validate	8	96	103

Field Descriptions:

- **Region** - The memory region.
- **Operation** - The parameter table operation to perform.

Parameters_Memory_Region_Release.T:

A packed record which holds the parameter memory region to release as well as the status returned from the parameter update operation.

Table 27: Parameters_Memory_Region_Release Packed Record : 104 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Region	Memory_ Region. T	-	96	0	95

Status	Parameter_Enums.Parameter_Table_Update_Status.E	0 => Uninitialized 1 => Success 2 => Length_Error 3 => Crc_Error 4 => Parameter_Error 5 => Dropped 6 => Individual_Parameter_Modified	8	96	103

Field Descriptions:

- **Region** - The memory region.
- **Status** - The return status from the parameter update operation.

Sys_Time.T:

A record which holds a time stamp using GPS format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 28: Sys_Time Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces.Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces.Unsigned_32	0 to 4294967295	32	32	63

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^32) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 29: Tick Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Count	Interfaces.Unsigned_32	0 to 4294967295	32	64	95

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Ccsds_Enums.Ccsds_Packet_Type.E:

This single bit is used to identify that this is a Telecommand Packet or a Telemetry Packet. A Telemetry Packet has this bit set to value 0; therefore, for all Telecommand Packets Bit 3 shall be set to value 1.

Table 30: Ccsds_Packet_Type Literals:

Name	Value	Description
Telemetry	0	Indicates a telemetry packet
Telecommand	1	Indicates a telecommand packet

Ccsds_Enums.Ccsds_Secondary_Header_Indicator.E:

This one bit flag signals the presence (Bit 4 = 1) or absence (Bit 4 = 0) of a Secondary Header data structure within the packet.

Table 31: Ccsds_Secondary_Header_Indicator Literals:

Name	Value	Description
Secondary_Header_Not_Present	0	Indicates that the secondary header is not present within the packet
Secondary_Header_Present	1	Indicates that the secondary header is present within the packet

Ccsds_Enums.Ccsds_Sequence_Flag.E:

This flag provides a method for defining whether this packet is a first, last, or intermediate component of a higher layer data structure.

Table 32: Ccsds_Sequence_Flag Literals:

Name	Value	Description
Continuationsegment	0	Continuation component of higher data structure
Firstsegment	1	First component of higher data structure
Lastsegment	2	Last component of higher data structure
Unsegmented	3	Standalone packet

Ccsds_Parameter_Table_Router_Enums.Table_Status.E:

Status of the most recent parameter table operation.

Table 33: Table_Status Literals:

Name	Value	Description
Receiving_Table	0	A FirstSegment has been received and the buffer is accumulating data.
Table_Update_Success	1	The table was successfully distributed to all destinations.

Table_Update_Failed	2	A downstream component rejected the table or timed out.
Unrecognized_Id	3	The completed table had a Table ID not found in the routing table.
Buffer_Overflow	4	The staging buffer overflowed while receiving table data.
Too_Small	5	A FirstSegment was too small to contain a Table ID.
Packet_Ignored	6	A packet was ignored (continuation/last without first, or unsegmented).

Command_Enums.Command_Response_Status.E:

This status enumeration provides information on the success/failure of a command through the command response connector.

Table 34: Command_Response_Status Literals:

Name	Value	Description
Success	0	Command was passed to the handler and successfully executed.
Failure	1	Command was passed to the handler not successfully executed.
Id_Error	2	Command id was not valid.
Validation_Error	3	Command parameters were not successfully validated.
Length_Error	4	Command length was not correct.
Dropped	5	Command overflowed a component queue and was dropped.
Register	6	This status is used to register a command with the command routing system.
Register_Source	7	This status is used to register command sender's source id with the command router for command response forwarding.

Parameter_Enums.Parameter_Table_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter table update.

Table 35: Parameter_Table_Update_Status Literals:

Name	Value	Description
Uninitialized	0	No table operation has been performed yet.
Success	1	Parameter table update operation was successful.
Length_Error	2	Parameter table length was not correct.
Crc_Error	3	The computed CRC of the table does not match the stored CRC.
Parameter_Error	4	An individual parameter was found invalid due to a constraint error within a component, or failing component-specific validation.

Dropped	5	The operation could not be performed because it was dropped from a full queue.
Individual_Parameter_Modified	6	An individual parameter was updated in the parameter table without updating the whole table. In this case, the reported CRC, version, and update time of the table can no longer be trusted to be accurate. This status will be reported when using the Update_Parameter command in the Parameters component, which is intended to be a backdoor, ground-use only command.

Parameter_Enums.Parameter_Table_Operation_Type.E:

This enumeration lists the different parameter table operations that can be performed.

Table 36: Parameter_Table_Operation_Type Literals:

Name	Value	Description
Get	0	Retrieve the current values of the parameters.
Set	1	Set the current values of the parameters.
Validate	2	Validate the current values of the parameters.